Cálculo Computacional e Estatística

Python para Operações Matemáticas

Python é uma linguagem de programação gerenciada pela Python Software Foundation. Essa linguagem é de alto nível disponível em código aberto (open source).

O que você pode fazer com Python?

- Acelerar o aprendizado do conteúdo de cálculo computacional
- Construir sistemas baseado em Web com frameworks
- Analisar dados
- Criar sistemas que acoplem em projetos de Inteligência Artificial
- Aplicar técnicas das bibliotecas em Machine Learning
- Facilitar a criação de aplicativos
- Construir sistemas para desktop

O Python possui **bibliotecas** padronizadas dotadas de **muitos recursos**, que atraem boa parte dos programadores.

A arquitetura do python possui frameworks e módulos que são implementados e atualizados pela comunidade que contribui com o projeto.

Em que o Python nos ajudará?

- Auxiliará o cientista da computação a resolver problemas reais utilizando os recursos computacionais.
- Facilitará a implementação da modelagem dos problemas para testar as soluções computacionais.
- Ajudará nas soluções computacionais por meio de algoritmos e sua correta execução.
- Suportará soluções via software de modo a garantir consistências nos algoritmos de cálculos em fluxos dinâmicos de grandes quantidades de dados.
- Suportará a disciplina de cálculo computacional.

O Python é uma linguagem de programação de alto nível (figura1). LP são linguagens normais, com um teor preciso e instruções, estas últimas, são executadas em um sistema computacional. Um código fonte é um conjunto de palavras associadas as regras específicas da linguagem que está sendo utilizada para a programação.

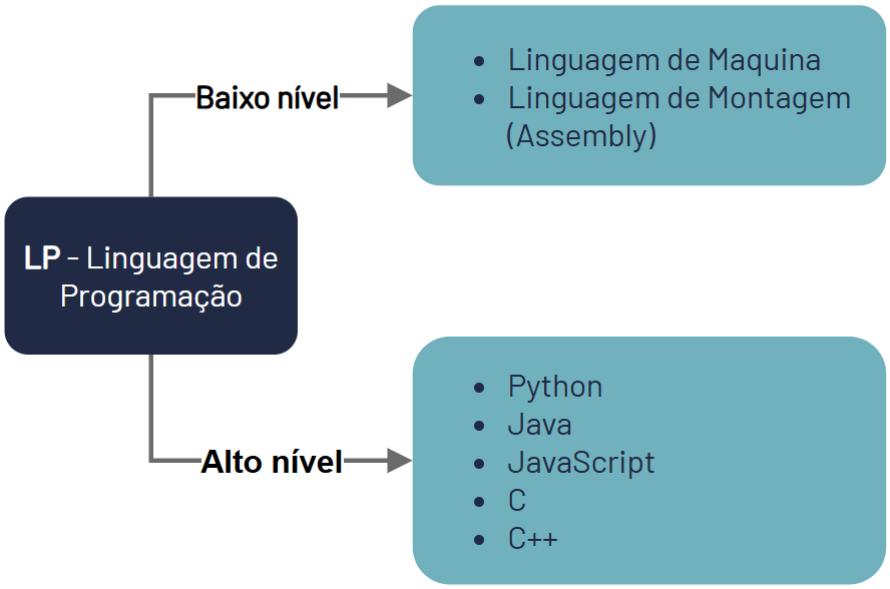


Figura 1 – Python é uma linguagem de alto nível (autoral)

Esse conjunto passa por um processo que traduz em uma linguagem reconhecida pelo computador. Isso pode ocorrer em três abordagens, a abordagem está relacionada a propriedade da sua implementação (figura 2).

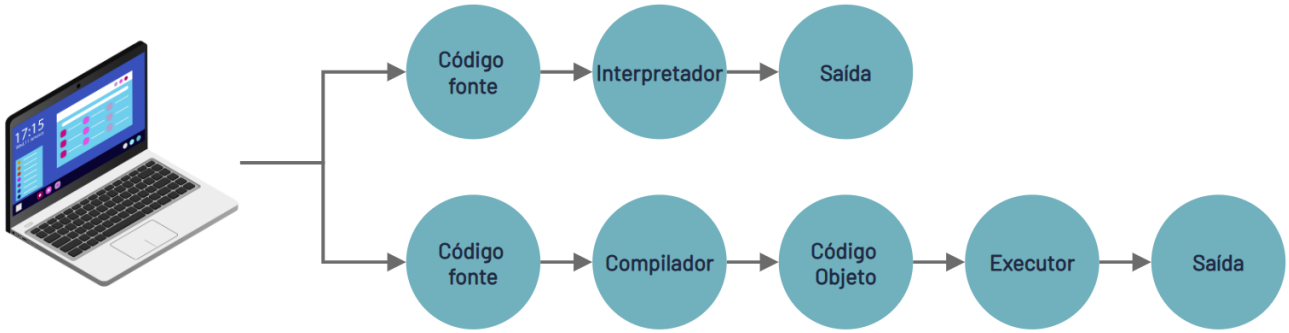


Figura 2 – Propriedades das implementações das abordagens das linguagens de programação. (autoral)

A abordagem da Linguagem Interpretada tem as seguintes características:

- É transformada em linguagem de máquina em tempo de execução do código.
- O Interpretador de código é carregado na memória da máquina para execução.
- A memória é gerenciada.

Explicando melhor

Na abordagem em linguagem interpretada, um código fonte, como um arquivo é escrito com determinados comandos relativos à linguagem de alto nível.
Esse código fonte é dado como entrada em outro programa, que é o interpretador. Esse interpretador segue linha a linha do arquivo fonte. Cada linha de código é interpretada e algo é executado internamente no computador. E pode gerar algum resultado na saída do terminal do computador, encaminhado para a impressora, algum tipo de bytes para a internet, dentre outras interações de saída.

A abordagem da linguagem compilada tem as seguintes características:

- Traduzir as instruções do código fonte em uma linguagem de máquina na etapa de compilação.
- O código executável gerado não interpreta a linguagem, apenas contém a linguagem de máquina.
- A execução é mais rápida do que linguagens interpretadas. O gerenciamento do uso da memória fico por conta do programador.

Explicando melhor

Na linguagem compilada, o código fonte, o programa, é para o compilador, esse compilador transforma o programa em código de máquina. Esse código de máquina são os zeros (0) e uns (1). Esse código objeto é gravado em disco como um arquivo executável. Posteriormente esse arquivo executável, depois que a compilação terminou, pode ser executado, por um programa executor, tipicamente é próprio sistema operacional. O sistema operacional vai executar esse arquivo "executável" que vai realizar alguma interação com o usuário ou gerar uma saída.

As linguagens com Byte-Code usam as duas abordagens, linguagens interpretadas e compiladas. Linguagens modernas combinam as duas abordagens como o Java e Python:

- No processo de compilação, é gerado um arquivo intermediário, denominado de bytecode, não gera uma linguagem de máquina. Antes de iniciar a execução do programa, um compilador traduz o código-fonte para bytecode, são códigos em bytes.
- No processo de execução, o byte-code é interpretado, gerando assim a linguagem de máquina. Ao iniciar a execução do programa, o interpretador lê os bytecodes um a um executando os comandos correspondentes.
- A interpretação do byte-code é realizada a primeira vez em que for executado, permitindo execuções posteriores mais rápidas.
- Nesta abordagem a memória é gerenciada.

Explicando melhor

Antes de iniciar do programa, o compilador traduz o código fonte desse programa, para um código de bytes, que chamamos de bytecodes. Os bytecodes são uma forma reduzida do mesmo programa só que de maneira mais fácil para que o computador possa interpretá-la. Após isso, o bytecode, arquivo pré-compilado, é submetido a um interpretador que lê eficientemente os bytecodes e o executando um a um os comandos correspondentes.

Vamos abrir o interpretador de comandos Python:

No terminal, faça:

```
Terminal
>>> a=1
>>> b=2
>>> c=a+b
>>> c
3
>>> a=4
>>> b=2
>>> c=a-b
>>> c
2
>>> a=3
>>> b=2
>>> c=a*b
>>> c
6
>>> a=8
>>> b=4
>>> c=a/b
>>> c
2.0
```

Cuidado com a indentação 1 em Python Vamos fazer um exemplo:

```
Terminal
>>>     a = 1
File "<stdin>", line 1
a=1
^
IndentationError: unexpected indent
```